

c SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 35%

Dr.G.Ayyappan

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	5
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	5
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.	BL3 – Apply	5
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	5
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	5
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	5
7	Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.	BL6 – Create	5

8	Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.	BL3 – Apply	5
9	Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.	BL3 – Apply	5
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	5

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method

Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

QUESTION 1 : Relational Algebra Operations on Employee and Department

Assume:

Employee

EmpID	EmpName	DeptID	Salary
101	Arun	10	60000
102	Priya	20	55000
103	Ravi	10	45000
104	Anu	30	70000

Department

DeptID	DeptName
10	IT
20	HR
30	Finance

Selection (σ)

Find employees whose salary is greater than 50,000:

σ Salary > 50000 (Employee)

Projection (π)

Display only employee names:

π EmpName (Employee)

Union ($\cup$)

If Employee_IT and Employee_HR have the same attributes:

Employee_IT $\cup$ Employee_HR

Difference ($-$)

Find employees who are not in HR:

Employee - Employee_HR

Cartesian Product (×)

Employee × Department

This combines every employee with every department.

Join (⋈)

Join Employee and Department using DeptID:

Employee ⋈ Employee.DeptID = Department.DeptID Department

This retrieves employee details along with department names.

QUESTION 2 : SQL DDL For STUDENT And DEPARTMENT

Create Table Department (

Deptid Int Primary Key,

Deptname Varchar(50) Not Null Unique

);

Create Table Student (

Sid Int Primary Key,

Name Varchar(100) Not Null,

Dob Date Not Null,

Deptid Int Not Null,

Foreign Key (Deptid) References Department(Deptid

);

Constraints used:

- PRIMARY KEY → uniquely identifies records.
- NOT NULL → prevents empty values.

- UNIQUE → prevents duplicate department names.
- FOREIGN KEY → maintains relationship between Student and Department.

```
mysql> DESC DEPARTMENT;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| DeptNo | varchar(4)    | NO   | PRI | NULL    |       |
| DeptName | varchar(15)  | YES  |     | NULL    |       |
| DeptHead | varchar(4)    | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.448 sec)

mysql> DESC STUDENT;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| RegNo | int           | YES  |     | NULL    |       |
| Name  | varchar(15)   | YES  |     | NULL    |       |
| Gender | char(1)       | YES  |     | NULL    |       |
| DOB   | date          | YES  |     | NULL    |       |
| MobileNo | bigint       | YES  |     | NULL    |       |
| City  | varchar(15)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.031 sec)
```

QUESTION 3 : GROUP BY, HAVING and Aggregate Functions

Assume **Sales** table:

Sales(SaleID, Product, Category, Quantity, Amount)

COUNT

Find number of sales in each category:

```
SELECT Category, COUNT(*) AS Total_Sales
FROM Sales
GROUP BY Category;
```

SUM

Find total sales amount for each category:

```
SELECT Category, SUM(Amount) AS Total_Amount
FROM Sales
GROUP BY Category;
```

AVG

Find average sales amount:

```
SELECT Category, AVG(Amount) AS Average_Amount
FROM Sales
GROUP BY Category;
```

MAX

Find maximum sale amount:

```
SELECT Category, MAX(Amount) AS Maximum_Amount
FROM Sales
GROUP BY Category;
```

MIN

Find minimum sale amount:

```
SELECT Category, MIN(Amount) AS Minimum_Amount
FROM Sales
GROUP BY Category;
```

HAVING

Find categories whose total sales exceed 50,000:

```
SELECT Category, SUM(Amount) AS Total_Amount
FROM Sales
GROUP BY Category
HAVING SUM(Amount) > 50000;
```

Remember:

WHERE filters individual rows, while HAVING filters groups.

```
mysql> SELECT * FROM Sales;
+-----+-----+-----+-----+-----+
| SaleID | Product | Category | Quantity | Amount |
+-----+-----+-----+-----+-----+
| 1 | Laptop | Electronics | 2 | 90000 |
| 2 | Mobile | Electronics | 3 | 45000 |
| 3 | Chair | Furniture | 4 | 20000 |
| 4 | Table | Furniture | 2 | 15000 |
| 5 | Pen | Stationery | 10 | 500 |
| 6 | Notebook | Stationery | 5 | 750 |
| 7 | Monitor | Electronics | 1 | 15000 |
| 8 | Keyboard | Electronics | 2 | 3000 |
| 9 | Desk | Furniture | 1 | 8000 |
| 10 | Pencil | Stationery | 20 | 400 |
+-----+-----+-----+-----+-----+
10 rows in set (0.001 sec)

mysql> █
```

4. Correlated Subquery – Salary Greater Than Department Average

Assume:

Employee(EmpID, EmpName, DeptID, Salary)

Query:

```
SELECT E.EmpID, E.EmpName, E.DeptID, E.Salary
FROM Employee E
WHERE E.Salary > (
    SELECT AVG(E2.Salary)
    FROM Employee E2
    WHERE E2.DeptID = E.DeptID
);
```

Explanation

The inner query calculates the **average salary of the employee's own department**.

The outer query then selects employees whose salary is greater than that department average.

```
4. UPDATE with JOIN - Increase salary by 10% for employees in 'IT' department
mysql> UPDATE Employee E
-> INNER JOIN Department D ON E.DeptID = D.DeptID
-> SET E.Salary = E.Salary * 1.10
-> WHERE D.DeptName = 'IT';
Query OK, 3 rows affected (0.002 sec)
Rows matched: 3  Changed: 3  Warnings: 0

mysql> SELECT * FROM Employee; -- Before Update
+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | Salary |
+-----+-----+-----+-----+
| 101   | Arun    | 10     | 60000   |
| 102   | Priya   | 20     | 45000   |
| 103   | Ravi    | 20     | 48000   |
| 104   | Anu     | 30     | 70000   |
| 105   | Kiran   | 30     | 47000   |
| 106   | Neha    | 10     | 65000   |
| 107   | Mohan   | 20     | 50000   |
| 108   | Pooja   | 30     | 58000   |
| 109   | Sahil   | 10     | 52000   |
+-----+-----+-----+-----+
9 rows in set (0.001 sec)

mysql> SELECT * FROM Employee; -- After Update
+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | Salary |
+-----+-----+-----+-----+
| 101   | Arun    | 10     | 66000.00 |
| 102   | Priya   | 20     | 45000.00 |
| 103   | Ravi    | 20     | 48000.00 |
| 104   | Anu     | 30     | 70000.00 |
| 105   | Kiran   | 30     | 47000.00 |
| 106   | Neha    | 10     | 71500.00 |
| 107   | Mohan   | 20     | 50000.00 |
| 108   | Pooja   | 30     | 58000.00 |
| 109   | Sahil   | 10     | 57200.00 |
+-----+-----+-----+-----+
9 rows in set (0.001 sec)
```

QUESTION 5. All Types of JOIN Operations

Assume:

Employee

Employee(EmpID, EmpName, ProjectID)

Project

Project(ProjectID, ProjectName)

1. INNER JOIN

Returns matching records from both tables.

```
SELECT E.EmpName, P.ProjectName
FROM Employee E
INNER JOIN Project P
ON E.ProjectID = P.ProjectID;
```

2. LEFT OUTER JOIN

Returns all employees and matching projects.

```
SELECT E.EmpName, P.ProjectName
FROM Employee E
LEFT JOIN Project P
ON E.ProjectID = P.ProjectID;
```

3. RIGHT OUTER JOIN

Returns all projects and matching employees.

```
SELECT E.EmpName, P.ProjectName
FROM Employee E
RIGHT JOIN Project P
ON E.ProjectID = P.ProjectID;
```

4. FULL OUTER JOIN

Returns all employees and all projects, whether matching or not.

```
SELECT E.EmpName, P.ProjectName
FROM Employee E
FULL OUTER JOIN Project P
ON E.ProjectID = P.ProjectID;
```

Note: MySQL does not directly support FULL OUTER JOIN. In MySQL, it can be simulated using LEFT JOIN, RIGHT JOIN, and UNION.

```
SELECT E.EmpName, P.ProjectName
FROM Employee E
LEFT JOIN Project P
ON E.ProjectID = P.ProjectID
```

UNION

```
SELECT E.EmpName, P.ProjectName
FROM Employee E
RIGHT JOIN Project P
ON E.ProjectID = P.ProjectID;
```

5. CROSS JOIN

Returns the Cartesian product.

```
SELECT E.EmpName, P.ProjectName
FROM Employee E
CROSS JOIN Project P;
```

1. INNER JOIN

mysql> SELECT E.EmpID, E.EmpName, P.ProjectID, P.ProjectName
-> FROM Employee E
-> INNER JOIN Project P ON E.ProjectID = P.ProjectID;

EmpID	EmpName	ProjectID	ProjectName
101	Arun	1	Website
102	Priya	2	Mobile App
104	Anu	3	AI System

3 rows in set (0.002 sec)

3. RIGHT OUTER JOIN

mysql> SELECT E.EmpID, E.EmpName, P.ProjectID, P.ProjectName
-> FROM Employee E
-> RIGHT JOIN Project P ON E.ProjectID = P.ProjectID;

EmpID	EmpName	ProjectID	ProjectName
101	Arun	1	Website
102	Priya	2	Mobile App
104	Anu	3	AI System
NULL	NULL	4	Data Mining

4 rows in set (0.002 sec)

5. CROSS JOIN

mysql> SELECT E.EmpID, E.EmpName, P.ProjectID, P.ProjectName
-> FROM Employee E
-> CROSS JOIN Project P;

EmpID	EmpName	ProjectID	ProjectName
101	Arun	1	Website
101	Arun	2	Mobile App
101	Arun	3	AI System
101	Arun	4	Data Mining
102	Priya	1	Website
102	Priya	2	Mobile App
102	Priya	3	AI System
102	Priya	4	Data Mining
103	Ravi	1	Website
103	Ravi	2	Mobile App
103	Ravi	3	AI System
103	Ravi	4	Data Mining
104	Anu	1	Website
104	Anu	2	Mobile App
104	Anu	3	AI System
104	Anu	4	Data Mining
105	Kiran	1	Website
105	Kiran	2	Mobile App
105	Kiran	3	AI System
105	Kiran	4	Data Mining

20 rows in set (0.002 sec)

2. LEFT OUTER JOIN

mysql> SELECT E.EmpID, E.EmpName, P.ProjectID, P.ProjectName
-> FROM Employee E
-> LEFT JOIN Project P ON E.ProjectID = P.ProjectID;

EmpID	EmpName	ProjectID	ProjectName
101	Arun	1	Website
102	Priya	2	Mobile App
103	Ravi	NULL	NULL
104	Anu	3	AI System
105	Kiran	NULL	NULL

5 rows in set (0.002 sec)

4. FULL OUTER JOIN (Using LEFT JOIN + UNION + RIGHT JOIN)

mysql> SELECT E.EmpID, E.EmpName, P.ProjectID, P.ProjectName
-> FROM Employee E LEFT JOIN Project P ON E.ProjectID = P.ProjectID
-> UNION
-> SELECT E.EmpID, E.EmpName, P.ProjectID, P.ProjectName
-> FROM Employee E RIGHT JOIN Project P ON E.ProjectID = P.ProjectID;

EmpID	EmpName	ProjectID	ProjectName
101	Arun	1	Website
102	Priya	2	Mobile App
103	Ravi	NULL	NULL
104	Anu	3	AI System
105	Kiran	NULL	NULL
NULL	NULL	4	Data Mining

6 rows in set (0.003 sec)

Employee Table

mysql> SELECT * FROM Employee;

EmpID	EmpName	ProjectID
101	Arun	1
102	Priya	2
103	Ravi	NULL
104	Anu	3
105	Kiran	NULL

5 rows in set (0.001 sec)

Project Table

mysql> SELECT * FROM Project;

ProjectID	ProjectName
1	Website
2	Mobile App
3	AI System
4	Data Mining

4 rows in set (0.001 sec)

QUESTION 6 : Create Deptsalaryview

Assume:

Employee(EmpID, EmpName, DeptID, Salary)
Department(DeptID, DeptName)

Create the view:

```
CREATE VIEW DeptSalaryView AS
SELECT
    D.DeptName,
    COUNT(E.EmpID) AS Total_Employees,
```

```

    AVG(E.Salary) AS Average_Salary
FROM Department D
LEFT JOIN Employee E
ON D.DeptID = E.DeptID
GROUP BY D.DeptID, D.DeptName;

```

Query the view

```

SELECT *
FROM DeptSalaryView;

```

Find Departments With Average Salary Greater Than 60,000

```

SELECT *
FROM DeptSalaryView
WHERE Average_Salary > 60000;

```

```

5. RIGHT OUTER JOIN
mysql> SELECT E.EmpID, E.EmpName, P.ProjectID, P.ProjectName
-> FROM Employee E
-> RIGHT JOIN Project P ON E.ProjectID = P.ProjectID;
+-----+-----+-----+-----+
| EmpID | EmpName | ProjectID | ProjectName |
+-----+-----+-----+-----+
| 101   | Arun    | 1         | Website     |
| 102   | Priya   | 2         | Mobile App  |
| 104   | Anu     | 3         | AI System   |
| NULL  | NULL    | 4         | Data Mining |
+-----+-----+-----+-----+
4 rows in set (0.002 sec)

5. FULL OUTER JOIN (Using LEFT JOIN + UNION + RIGHT JOIN)
mysql> SELECT E.EmpID, E.EmpName, P.ProjectID, P.ProjectName
-> FROM Employee E LEFT JOIN Project P ON E.ProjectID = P.ProjectID
-> UNION
-> SELECT E.EmpID, E.EmpName, P.ProjectID, P.ProjectName
-> FROM Employee E RIGHT JOIN Project P ON E.ProjectID = P.ProjectID;
+-----+-----+-----+-----+
| EmpID | EmpName | ProjectID | ProjectName |
+-----+-----+-----+-----+
| 101   | Arun    | 1         | Website     |
| 102   | Priya   | 2         | Mobile App  |
| 103   | Ravi    | NULL      | NULL        |
| 104   | Anu     | 3         | AI System   |
| 105   | Kiran   | NULL      | NULL        |
| NULL  | NULL    | 4         | Data Mining |
+-----+-----+-----+-----+
6 rows in set (0.003 sec)

```

QUESTION 7 : Relational Algebra – Students Enrolled In All Courses

Assume:

```

Student(SID, Name)
Enrollment(SID, CourseID)
Course(CourseID, CourseName)

```

We need to find students who are enrolled in **every course offered**.

Using **division** ($\div$):

First obtain the student-course relationship:

$R = \pi_{SID, CourseID}(Enrollment)$

Then:

$R \div \pi \text{ CourseID (Course)}$

Finally, join with Student to obtain names:

$\pi \text{ Name (Student} \bowtie (\pi \text{ SID, CourseID(Enrollment)} \div \pi \text{ CourseID(Course)))}$

Final expression:

$\pi \text{ Name (Student} \bowtie (\pi \text{ SID, CourseID(Enrollment)} \div \pi \text{ CourseID(Course)))}$

This returns students who have enrolled in **all courses**.

QUESTION 8 : SQL DML – INSERT, UPDATE, DELETE with Constraints

Assume:

DEPARTMENT(DepartmentID, DeptName)
STUDENT(SID, Name, DOB, DepartmentID)

INSERT

First insert the parent department:

```
INSERT INTO DEPARTMENT  
VALUES (10, 'IT');
```

Then insert a student:

```
INSERT INTO STUDENT  
VALUES (101, 'Arun', '2004-05-15', 10);
```

The foreign key ensures that DepartmentID = 10 exists in DEPARTMENT.

UPDATE

Update student's department:

```
UPDATE STUDENT  
SET DepartmentID = 10  
WHERE SID = 101;
```

The new department ID must exist in DEPARTMENT.

DELETE

Delete a student:

```
DELETE FROM STUDENT  
WHERE SID = 101;
```

Delete a department

```
DELETE FROM DEPARTMENT  
WHERE DeptID = 10;
```

If students still reference department 10, the deletion will normally be rejected because of the foreign key constraint.

```
8. SQL DML - INSERT, UPDATE, DELETE with Integrity Constraints  
mysql> -- Insert (Department first)  
mysql> INSERT INTO DEPARTMENT VALUES (10, 'IT'), (20, 'CSE'),  
(30, 'ECE');  
Query OK, 3 rows affected (0.002 sec)  
Records: 3 Duplicates: 0 Warnings: 0  
  
mysql> SELECT * FROM DEPARTMENT;  
+-----+-----+  
| DeptID | DeptName |  
+-----+-----+  
| 10     | IT       |  
| 20     | CSE      |  
| 30     | ECE      |  
+-----+-----+  
3 rows in set (0.000 sec)  
  
mysql> -- Insert (Student)  
mysql> INSERT INTO STUDENT VALUES  
-> (101, 'Arun', '2004-05-15', 10),  
-> (102, 'Priya', '2004-08-20', 20),  
-> (103, 'Ravi', '2004-12-10', 10);  
Query OK, 3 rows affected (0.001 sec)  
Records: 3 Duplicates: 0 Warnings: 0  
  
mysql> SELECT * FROM STUDENT;  
+-----+-----+-----+-----+  
| SID | Name | DOB       | DeptID |  
+-----+-----+-----+-----+  
| 101 | Arun | 2004-05-15 | 10     |  
| 102 | Priya | 2004-08-20 | 20     |  
| 103 | Ravi | 2004-12-10 | 10     |  
+-----+-----+-----+-----+  
3 rows in set (0.000 sec)  
  
mysql> -- Update (Change DeptID of SID = 102)  
mysql> UPDATE STUDENT SET DeptID = 30 WHERE SID = 102;  
Query OK, 1 row affected (0.001 sec)  
Rows matched: 1 Changed: 1 Warnings: 0  
  
mysql> SELECT * FROM STUDENT;  
+-----+-----+-----+-----+  
| SID | Name | DOB       | DeptID |  
+-----+-----+-----+-----+  
| 101 | Arun | 2004-05-15 | 10     |  
| 102 | Priya | 2004-08-20 | 30     |  
| 103 | Ravi | 2004-12-10 | 10     |  
+-----+-----+-----+-----+  
3 rows in set (0.000 sec)  
  
mysql> -- Delete (Remove a student)  
mysql> DELETE FROM STUDENT WHERE SID = 103;  
Query OK, 1 row affected (0.001 sec)  
Rows matched: 1 Changed: 1 Warnings: 0  
  
mysql> SELECT * FROM STUDENT;  
+-----+-----+-----+-----+  
| SID | Name | DOB       | DeptID |  
+-----+-----+-----+-----+  
| 101 | Arun | 2004-05-15 | 10     |  
| 102 | Priya | 2004-08-20 | 30     |  
+-----+-----+-----+-----+  
2 rows in set (0.000 sec)  
  
mysql> -- Delete (Remove department)  
mysql> DELETE FROM DEPARTMENT WHERE DeptID = 20;  
Query OK, 1 row affected (0.001 sec)  
Rows matched: 1 Changed: 1 Warnings: 0  
  
mysql> SELECT * FROM DEPARTMENT;  
+-----+-----+  
| DeptID | DeptName |  
+-----+-----+  
| 10     | IT       |  
| 30     | ECE      |  
+-----+-----+  
2 rows in set (0.000 sec)
```

QUESTION 9 : Tuple Relational Calculus

Question:

Find all employees who work in the IT department and earn more than 50,000.

Assume:

Employee(EmpID, EmpName, DeptID, Salary)
Department(DeptID, DeptName)

Tuple relational calculus expression:

$\{ e \mid \text{Employee}(e) \wedge \exists d (\text{Department}(d) \wedge e.\text{DeptID} = d.\text{DeptID} \wedge d.\text{DeptName} = \text{'IT'} \wedge e.\text{Salary} > 50000) \}$

Meaning:

- e represents an employee tuple.
- d represents a department tuple.
- Employee's DeptID must match Department's DeptID.
- Department must be IT.
- Salary must be greater than 50000.

```

9. TUPLE RELATIONAL CALCULUS - Employees in 'IT' department
   earning more than 50000
mysql> SELECT E.EmpID, E.EmpName, E.DeptID, E.Salary
-> FROM Employee E
-> WHERE EXISTS (SELECT *
->                FROM Department D
->                WHERE E.DeptID = D.DeptID
->                AND D.DeptName = 'IT'
->                AND E.Salary > 50000);
+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | Salary |
+-----+-----+-----+-----+
| 101   | Arun    | 10     | 60000  |
| 106   | Neha    | 10     | 65000  |
| 109   | Sahil   | 10     | 52000  |
+-----+-----+-----+-----+
3 rows in set (0.001 sec)

mysql> SELECT * FROM Department;
+-----+-----+
| DeptID | DeptName |
+-----+-----+
| 10     | IT        |
| 20     | HR        |
| 30     | Finance  |
+-----+-----+
3 rows in set (0.000 sec)

mysql> SELECT * FROM Employee;
+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | Salary |
+-----+-----+-----+-----+
| 101   | Arun    | 10     | 60000  |
| 102   | Priya   | 20     | 45000  |
| 103   | Ravi    | 20     | 48000  |
| 104   | Anu     | 30     | 70000  |
| 105   | Kiran   | 30     | 47000  |
| 106   | Neha    | 10     | 65000  |
| 107   | Mohan   | 20     | 50000  |
| 108   | Pooja   | 30     | 58000  |
| 109   | Sahil   | 10     | 52000  |
+-----+-----+-----+-----+
9 rows in set (0.001 sec)

```

QUESTION 10 : Airline Reservation System

Relational Schema

A simple airline reservation system can contain the following tables:

Passenger

Passenger(
 PassengerID PRIMARY KEY,
 PassengerName,
 Email,
 Phone
)

AIRPORT

```
Airport(  
    AirportID PRIMARY KEY,  
    AirportName,  
    City  
)
```

FLIGHT

```
Flight(  
    FlightID PRIMARY KEY,  
    FlightNumber,  
    SourceAirport,  
    DestinationAirport,  
    FlightDate,  
    DepartureTime,  
    ArrivalTime  
)
```

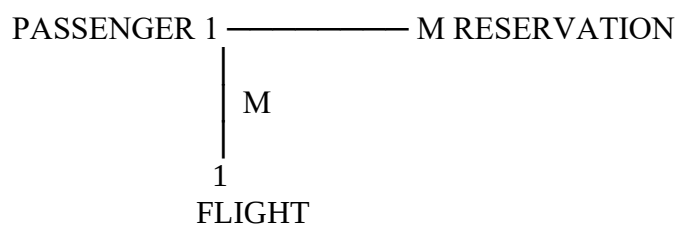
RESERVATION

```
Reservation(  
    ReservationID PRIMARY KEY,  
    PassengerID,  
    FlightID,  
    SeatNo,  
    BookingDate,  
    Status  
)
```

PAYMENT

```
Payment(
    PaymentID PRIMARY KEY,
    ReservationID,
    Amount,
    PaymentDate,
    PaymentStatus
)
```

Relationships



RESERVATION 1 ————— 1 PAYMENT

AIRPORT 1 ————— M FLIGHT

AIRPORT 1 ————— M FLIGHT

SQL Queries For Airline Reservation System

Query 1 – Join Passenger and Reservation

```
SELECT
    P.PassengerName,
    R.ReservationID,
    R.SeatNo,
    R.BookingDate
FROM Passenger P
INNER JOIN Reservation R
ON P.PassengerID = R.PassengerID;
```

Query 2 – Join Passenger, Reservation And Flight

```
SELECT
    P.PassengerName,
    F.FlightNumber,
    F.SourceAirport,
    F.DestinationAirport,
    R.SeatNo
FROM Passenger P
JOIN Reservation R
ON P.PassengerID = R.PassengerID
JOIN Flight F
ON R.FlightID = F.FlightID;
```

Query 3 – Subquery to Find Passengers Who Have Reservations

```
SELECT PassengerName
FROM Passenger
WHERE PassengerID IN (
    SELECT PassengerID
    FROM Reservation
);
```

Query 4 – Find Flights With More Than 5 Reservations

```
SELECT FlightID, COUNT(*) AS Total_Reservations
FROM Reservation
GROUP BY FlightID
HAVING COUNT(*) > 5;
```

Query 5 – Create and Query a View

Create a view showing passenger and flight details:

```
CREATE VIEW ReservationView AS
SELECT
    P.PassengerName,
    F.FlightNumber,
    F.SourceAirport,
    F.DestinationAirport,
    R.SeatNo,
    R.Status
FROM Passenger P
JOIN Reservation R
ON P.PassengerID = R.PassengerID
JOIN Flight F
ON R.FlightID = F.FlightID;
```

Query the view:

```
SELECT *
FROM ReservationView;
```

```
10. SUB-QUERY - Employees earning more than average salary
mysql> SELECT Avg(Salary) AS AverageSalary FROM Employee;
+-----+
| AverageSalary |
+-----+
| 54000.00      |
+-----+
1 row in set (0.001 sec)

mysql> SELECT EmpID, EmpName, DeptID, Salary
-> FROM Employee
-> WHERE Salary > (SELECT AVG(Salary) FROM Employee);
+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | Salary |
+-----+-----+-----+-----+
| 101   | Arun    | 10     | 60000  |
| 104   | Anu     | 30     | 70000  |
| 106   | Neha    | 10     | 65000  |
| 108   | Pooja   | 30     | 58000  |
+-----+-----+-----+-----+
4 rows in set (0.002 sec)

mysql> SELECT * FROM Employee;
+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | Salary |
+-----+-----+-----+-----+
| 101   | Arun    | 10     | 60000  |
| 102   | Priya   | 20     | 45000  |
| 103   | Ravi    | 20     | 48000  |
| 104   | Anu     | 30     | 70000  |
| 105   | Kiran   | 30     | 47000  |
| 106   | Neha    | 10     | 65000  |
| 107   | Mohan   | 20     | 50000  |
| 108   | Pooja   | 30     | 58000  |
| 109   | Sahil   | 10     | 52000  |
+-----+-----+-----+-----+
9 rows in set (0.001 sec)
```